Q3. [10 pts] Gradient Descent for k-means Clustering

Recall the loss function for k-means clustering with k clusters, sample points $x_1, ..., x_n$, and centers $\mu_1, ..., \mu_k$:

$$L = \sum_{j=1}^{k} \sum_{x_i \in S_j} \|x_i - \mu_j\|^2,$$

where S_j refers to the set of data points that are closer to μ_j than to any other cluster mean.

(1) [4 pts] Instead of updating μ_j by computing the mean, let's minimize L with **batch** gradient descent while holding the sets S_j fixed. Derive the update formula for μ_1 with learning rate (step size) ϵ.

$$\begin{aligned} \frac{\partial L}{\partial \mu_1} &= \frac{\partial}{\partial \mu_1} \sum_{x_i \in S_1} (x_i - \mu_1)^\top (x_i - \mu_1) \\ &= \sum_{x_i \in S_1} 2(\mu_1 - x_i). \end{aligned}$$

Therefore the update formula is

$$\mu_1 \leftarrow \mu_1 + \epsilon \sum_{x_i \in S_1} (x_i - \mu_1).$$

(Note: writing 2ϵ instead of ϵ is fine.)

(2) [2 pts] Derive the update formula for μ_1 with **stochastic** gradient descent on a single sample point x_i. Use learning rate ϵ.

$\mu_1 \leftarrow \mu_1 + \epsilon(x_i - \mu_1)$ if $x_i \in S_1$, otherwise no change.

(3) [4 pts] In this part, we will connect the batch gradient descent update equation with the standard k-means algorithm. Recall that in the update step of the standard algorithm, we assign each cluster center to be the mean (centroid) of the data points closest to that center. It turns out that a particular choice of the learning rate ϵ (which may be different for each cluster) makes the two algorithms (batch gradient descent and the standard k-means algorithm) have identical update steps. Let's focus on the update for the first cluster, with center μ_1. Calculate the value of ϵ so that both algorithms perform the same update for μ_1. (If you do it right, the answer should be very simple.)

In the standard algorithm, we assign $\mu_1 \leftarrow \sum_{x_i \in S_1} \frac{1}{|S_1|} x_i$.

Comparing to the answer in (1), we set $\sum_{x_i \in S_1} \frac{1}{|S_1|} x_i = \mu_1 + \epsilon \sum_{x_i \in S_1} (x_i - \mu_1)$ and solve for ϵ.

$$\begin{aligned} \sum_{x_i \in S_1} \frac{1}{|S_1|} x_i - \sum_{x_i \in S_1} \frac{1}{|S_1|} \mu_1 &= \epsilon \sum_{x_i \in S_1} (x_i - \mu_1) \\ \sum_{x_i \in S_1} \frac{1}{|S_1|} (x_i - \mu_1) &= \epsilon \sum_{x_i \in S_1} (x_i - \mu_1). \end{aligned}$$

Thus $\epsilon = \frac{1}{|S_1|}$.

(Note: answers that differ by a constant factor are fine if consistent with answer for (1).)

Q4. [10 pts] Kernels

(1) [2 pts] What is the primary motivation for using the kernel trick in machine learning algorithms?

If we want to map sample points to a very high-dimensional feature space, the kernel trick can save us from having to compute those features explicitly, thereby saving a lot of time.

(Alternative solution: the kernel trick enables the use of infinite-dimensional feature spaces.)

(2) [4 pts] Prove that for every design matrix $X \in \mathbb{R}^{n \times d}$, the corresponding kernel matrix is positive semidefinite.

For every vector $\mathbf{z} \in \mathbb{R}^n$,

$$\mathbf{z}^\top K \mathbf{z} = \mathbf{z}^\top X X^\top \mathbf{z} = |X^\top \mathbf{z}|^2,$$

which is clearly nonnegative.

(3) [2 pts] Suppose that a regression algorithm contains the following line of code.

$$\mathbf{w} \leftarrow \mathbf{w} + X^\top M X X^\top \mathbf{u}$$

Here, $X \in \mathbb{R}^{n \times d}$ is the design matrix, $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $M \in \mathbb{R}^{n \times n}$ is a matrix unrelated to X, and $\mathbf{u} \in \mathbb{R}^n$ is a vector unrelated to X. We want to derive a dual version of the algorithm in which we express the weights $\mathbf{w}$ as a linear combination of samples X_i (rows of X) and a dual weight vector $\mathbf{a}$ contains the coefficients of that linear combination. Rewrite the line of code in its dual form so that it updates $\mathbf{a}$ correctly (and so that $\mathbf{w}$ does not appear).

$$\mathbf{a} \leftarrow \mathbf{a} + M X X^\top \mathbf{u}$$

(4) [2 pts] Can this line of code for updating $\mathbf{a}$ be kernelized? If so, show how. If not, explain why.

Yes:

$$\mathbf{a} \leftarrow \mathbf{a} + M K \mathbf{u}$$

Q5. [12 pts] Let's PCA

You are given a design matrix $X = \begin{bmatrix} 6 & -4 \\ -3 & 5 \\ -2 & 6 \\ 7 & -3 \end{bmatrix}$. Let's use PCA to reduce the dimension from 2 to 1.

(1) [6 pts] Compute the covariance matrix for the sample points. (Warning: Observe that X is not centered.) Then compute the **unit** eigenvectors, and the corresponding eigenvalues, of the covariance matrix. *Hint:* If you graph the points, you can probably guess the eigenvectors (then verify that they really are eigenvectors).

The covariance matrix is $X^\top X = \begin{bmatrix} 82 & -80 \\ -80 & 82 \end{bmatrix}$.

Its unit eigenvectors are $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$ with eigenvalue 2 and $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{bmatrix}$ with eigenvalue 162. (Note: either eigenvector can be replaced with its negation.)

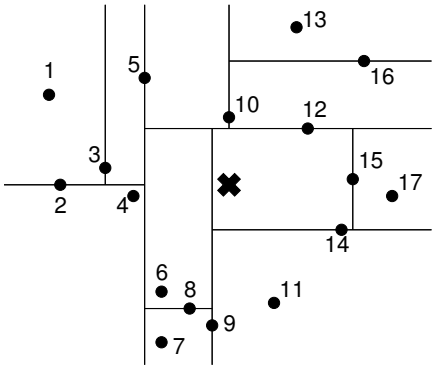
(2) [3 pts] Suppose we use PCA to project the sample points onto a one-dimensional space. What one-dimensional subspace are we projecting onto? For each of the four sample points in X (not the centered version of X!), write the coordinate (in principal coordinate space, not in $\mathbb{R}^2$) that the point is projected to.

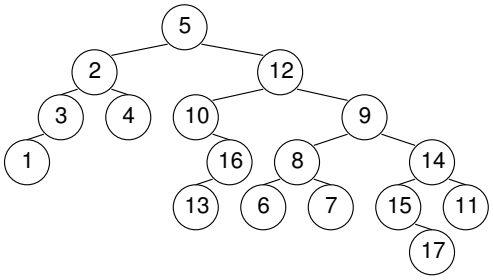
We are projecting onto the subspace spanned by $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{bmatrix}$. (Equivalently, onto the space spanned by $\begin{bmatrix} 1 \\ -1 \end{bmatrix}$. Equivalently, onto the line $x + y = 0$.) The projections are $(6, -4) \to \frac{10}{\sqrt{2}}$, $(-3, 5) \to -\frac{8}{\sqrt{2}}$, $(-2, 6) \to -\frac{8}{\sqrt{2}}$, $(7, -3) \to \frac{10}{\sqrt{2}}$.

(3) [3 pts] Given a design matrix X that is taller than it is wide, prove that every right singular vector of X with singular value σ is an eigenvector of the covariance matrix with eigenvalue σ^2.

If v is a right singular vector of X, then there is a singular value decomposition $X = UDV^\top$ such that v is a column of V. Here each of U and V has orthonormal columns, V is square, and D is square and diagonal. The covariance matrix is $X^\top X = VDU^\top UDV^\top = VD^2V^\top$. This is an eigendecomposition of $X^\top X$, so each singular vector in V with singular value σ is an eigenvector of $X^\top X$ with eigenvalue σ^2.

Q6. [10 pts] Trees





(1) [5 pts] Above, we have two depictions of the same k-d tree, which we have built to solve nearest neighbor queries. Each node of the tree at right represents a rectangular box at left, and also stores one of the sample points that lie inside that box. (The root node represents the whole plane $\mathbb{R}^2$.) If a treenode stores sample point i, then the line passing through point i (in the diagram at left) determines which boxes the child treenodes represent.

Simulate running an exact 1-nearest neighbor query, where the bold X is the query point. Recall that the query algorithm visits the treenodes in a smart order, and keeps track of the nearest point it has seen so far.

- Write down the numbers of all the sample points that serve as the "nearest point seen so far" sometime while the query algorithm is running, in the order they are encountered.
- Circle all the subtrees in the k-d tree at upper right that are never visited during this query. (This is why k-d tree search is usually faster than exhaustive search.)

Nearest point seen so far: first 5, then 12, then 10.

The unvisited subtrees are rooted at 2, 13, 7, and 17.

(2) [5 pts] We are building a decision tree for a 2-class classification problem. We have n training points, each having d real-valued features. At each node of the tree, we try every possible univariate split (i.e. for each feature, we try every possible splitting value for that feature) and choose the split that maximizes the information gain.

Explain why it is possible to build the tree in $O(ndh)$ time, where h is the depth of the tree's deepest node. Your explanation should include an analysis of the time to choose one node's split. Assume that we can radix sort real numbers in linear time.

Consider choosing the split at a node whose box contains n' sample points. For each of the d features, we can sort the sample points in $O(n'd)$ time. Then we can compute the entropy for the first split (separating the first sample in the sorted list from the others) in $O(n')$ time, then we can walk through the list and update the entropy for each successive split in $O(1)$ time, summing to a total of $O(n')$ time for each of the d features. So it takes $O(n'd)$ time overall to choose a split.

Each sample point participates in at most h treenodes, so each sample point contributes at most dh to the running time, for a total running time of at most $O(ndh)$.

Q7. [10 pts] Self-Driving Cars and Backpropagation

You want to train a neural network to drive a car. Your training data consists of grayscale 64×64 pixel images. The training labels include the human driver's steering wheel angle in degrees and the human driver's speed in miles per hour. Your neural network consists of an input layer with $64 \times 64 = 4{,}096$ units, a hidden layer with 2,048 units, and an output layer with 2 units (one for steering angle, one for speed). You use the ReLU activation function for the hidden units and no activation function for the outputs (or inputs).

(1) [2 pts] Calculate the number of parameters (weights) in this network. You can leave your answer as an expression. Be sure to account for the bias terms.

$$4097 \times 2048 + 2049 \times 2$$

(2) [3 pts] You train your network with the cost function $J = \frac{1}{2}|\mathbf{y} - \mathbf{z}|^2$. Use the following notation.

- $\mathbf{x}$ is a training image (input) vector with a 1 component appended to the end, $\mathbf{y}$ is a training label (input) vector, and $\mathbf{z}$ is the output vector. All vectors are column vectors.
- $r(\gamma) = \max\{0, \gamma\}$ is the ReLU activation function, $r'(\gamma)$ is its derivative (1 if $\gamma > 0$, 0 otherwise), and $r(\mathbf{v})$ is $r(\cdot)$ applied component-wise to a vector.
- $\mathbf{g}$ is the vector of hidden unit values before the ReLU activation functions are applied, and $\mathbf{h} = r(\mathbf{g})$ is the vector of hidden unit values after they are applied (but we append a 1 component to the end of $\mathbf{h}$).
- V is the weight matrix mapping the input layer to the hidden layer; $\mathbf{g} = V\mathbf{x}$.
- W is the weight matrix mapping the hidden layer to the output layer; $\mathbf{z} = W\mathbf{h}$.

Derive $\partial J / \partial W_{ij}$.

$$\begin{aligned} \frac{\partial J}{\partial W_{ij}} &= (\mathbf{z} - \mathbf{y})^\top \frac{\partial \mathbf{z}}{\partial W_{ij}} \\ &= (\mathbf{z}_i - \mathbf{y}_i)\mathbf{h}_j \end{aligned}$$

(3) [1 pt] Write $\partial J / \partial W$ as an outer product of two vectors. $\partial J / \partial W$ is a matrix with the same dimensions as W; it's just like a gradient, except that W and $\partial J / \partial W$ are matrices rather than vectors.

$$\frac{\partial J}{\partial W} = (\mathbf{z} - \mathbf{y})\mathbf{h}^\top$$

(4) [4 pts] Derive $\partial J / \partial V_{ij}$.

$$\begin{aligned} \frac{\partial J}{\partial V_{ij}} &= (\mathbf{z} - \mathbf{y})^\top \frac{\partial \mathbf{z}}{\partial V_{ij}} \\ &= (\mathbf{z} - \mathbf{y})^\top W \frac{\partial \mathbf{h}}{\partial V_{ij}} \\ &= (\mathbf{z} - \mathbf{y})^\top W \left[0, \ldots, r'(\mathbf{g}_i)\, \mathbf{x}_j, \ldots, 0\right]^\top \\ &= \left((\mathbf{z} - \mathbf{y})^\top W\right)_i r'(\mathbf{g}_i)\, \mathbf{x}_j. \end{aligned}$$